

資料結構

Data Structure

Lab 12

姓名： 黃聖傑

學號： 114310140

Lab12-Q1.

Sort the products according to the rules specified above and output the products from the most prominent position to the least prominent position. You may choose and implement any algorithm to solve this problem.

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
using namespace std;

// 商品結構
struct Product {
    string name;
    int expiry;    // 有效期限 (天)
    int popularity; // 熱銷程度 (1~10)

    // 自訂比較規則
    bool operator<(const Product& other) const {
        if (expiry != other.expiry)
            return expiry < other.expiry; // 有效期限越小越優先
        return popularity > other.popularity; // 熱銷程度越高越優先
    }
};

void quickSort(vector<Product>& products, int left, int right)
{
    if (left >= right) { // 基本情況：子陣列長度為 0 或 1
        return;
    }

    Product pivot = products[left]; // 選擇第一個元素作為基準
    int i = left; // i 從左邊開始
```

```

int j = right + 1; // j 從右邊開始
// 分割
while (true) {
    // 找到第一個大於或等於 pivot 的元素
    do {
        ++i;
    } while (i <= right && products[i] < pivot);
    // 找到第一個小於或等於 pivot 的元素
    do {
        --j;
    } while (pivot < products[j]);
    // 如果 i 和 j 相遇或交錯，分割完成
    if (i >= j) {
        break;
    }
    // 交換 i 和 j 位置的元素
    swap(products[i], products[j]);
}
// 將 pivot 放到正確的位置
swap(products[left], products[j]);
quickSort(products, left, j - 1);
quickSort(products, j + 1, right);
}

int main() {
    ifstream infile("input1.txt");
    if (!infile) {
        cerr << "無法打開檔案" << endl;
        return 1;
    }

    int N;
    infile >> N;
    infile.ignore(); // 忽略換行
    // 讀取商品資料(名稱、有效期限、熱銷程度)
    vector<Product> products;

```

```

for (int i = 0; i < N; ++i) {
    string line;
    getline(infile, line);

    // 從後往前提取最後兩個數字
    istringstream iss(line);
    string word;
    vector<string> tokens;

    while (iss >> word) {
        tokens.push_back(word);
    }

    if (tokens.size() < 3) continue;

    int popularity = stoi(tokens[tokens.size() - 1]);
    int expiry = stoi(tokens[tokens.size() - 2]);

    // 其餘部分是商品名稱
    string name = "";
    for (size_t j = 0; j < tokens.size() - 2; ++j) {
        if (j > 0) name += " ";
        name += tokens[j];
    }

    products.push_back({ name, expiry, popularity });
}

infile.close();
// 排序
if (!products.empty()) {
    quickSort(products, 0, static_cast<int>(products.size()) - 1);
}

// 輸出
cout << "商品排序:" << endl;
for (const Product& product : products) {
    cout << product.name << endl;
}

```

```
}

return 0;

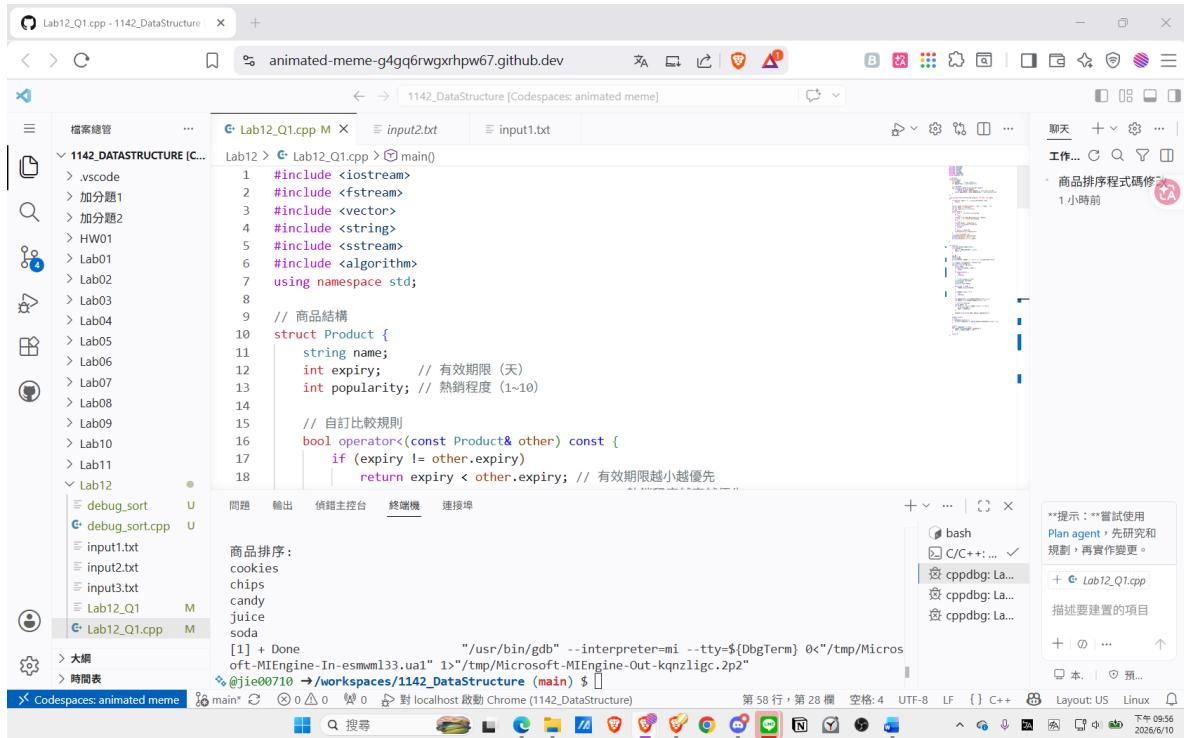
}
```

Result

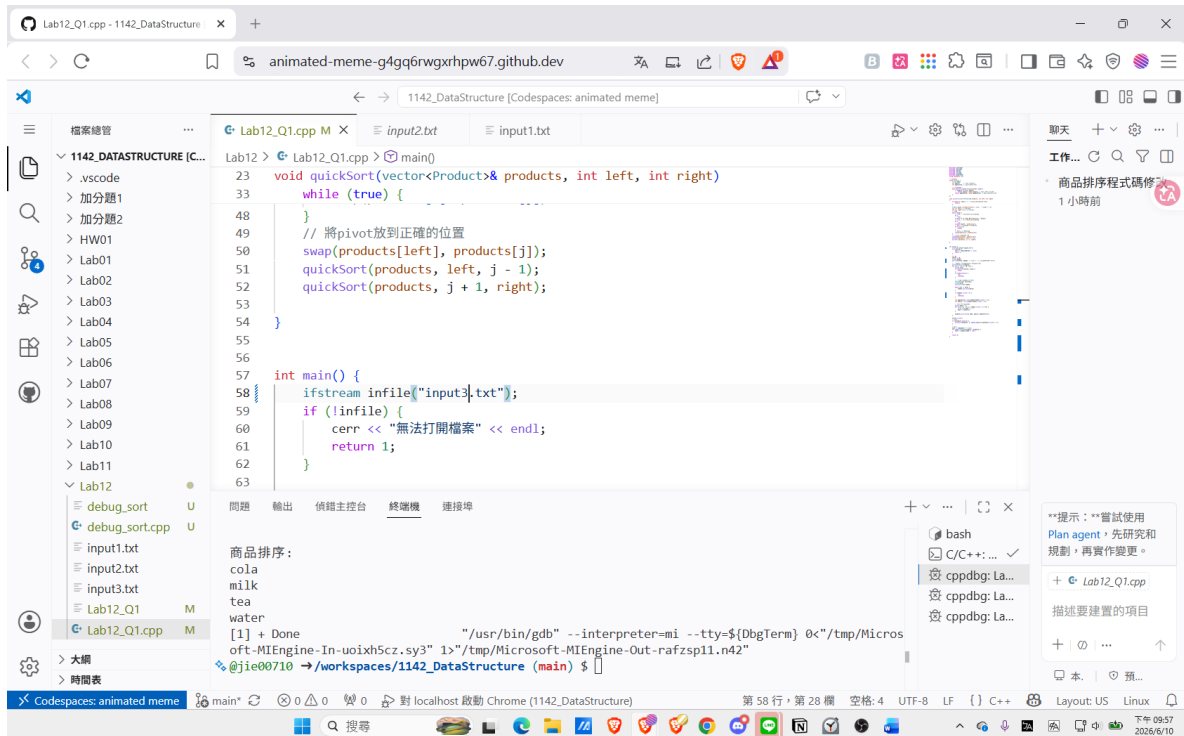
```
Lab12_Q1.cpp - 1142_DataStructure | X +
animated-meme-g4gq6rwxrhpw67.github.dev
1142_DataStructure [Codespaces: animated meme]
Lab12_Q1.cpp X input2.txt input1.txt
Lab12 > Lab12_Q1.cpp > main()
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <string>
5 #include <sstream>
6 #include <algorithm>
7 using namespace std;
8
9 // 商品結構
10 struct Product {
11     string name;
12     int expiry; // 有效期限 (天)
13     int popularity; // 熱銷程度 (1~10)
14
15     // 自訂比較規則
16     bool operator<(const Product& other) const {
17         if (expiry != other.expiry)
18             return expiry < other.expiry; // 有效期限越小越優先
19     }
20 };
21
22 int main() {
23     // 讀取 input1.txt 和 input2.txt 的內容
24     // ... (code continues) ...
25
26     // 輸出結果
27     cout << "商品排序:" << endl;
28     for (const auto& p : products) {
29         cout << p.name << endl;
30     }
31     return 0;
32 }
```

商品排序:
bread
milk
cola
instant noodles
chocolate
cookie

商品排序：
bread
milk
cola
instant noodles
chocolate
cookie



商品排序：
cookies
chips
candy
juice
soda



商品排序：

cola

milk

tea

water

Discussion

Algorithm : Quick sort

Time complexity : 平均 : $O(n \log n)$

Worst case : $O(n^2)$

Why : 在不確定數據分布及個數時 Quick sort 有較好的穩定度，不因為資料分散或集中而大幅提升運算時間